

Block Different Shape Sorting

1. Function Description

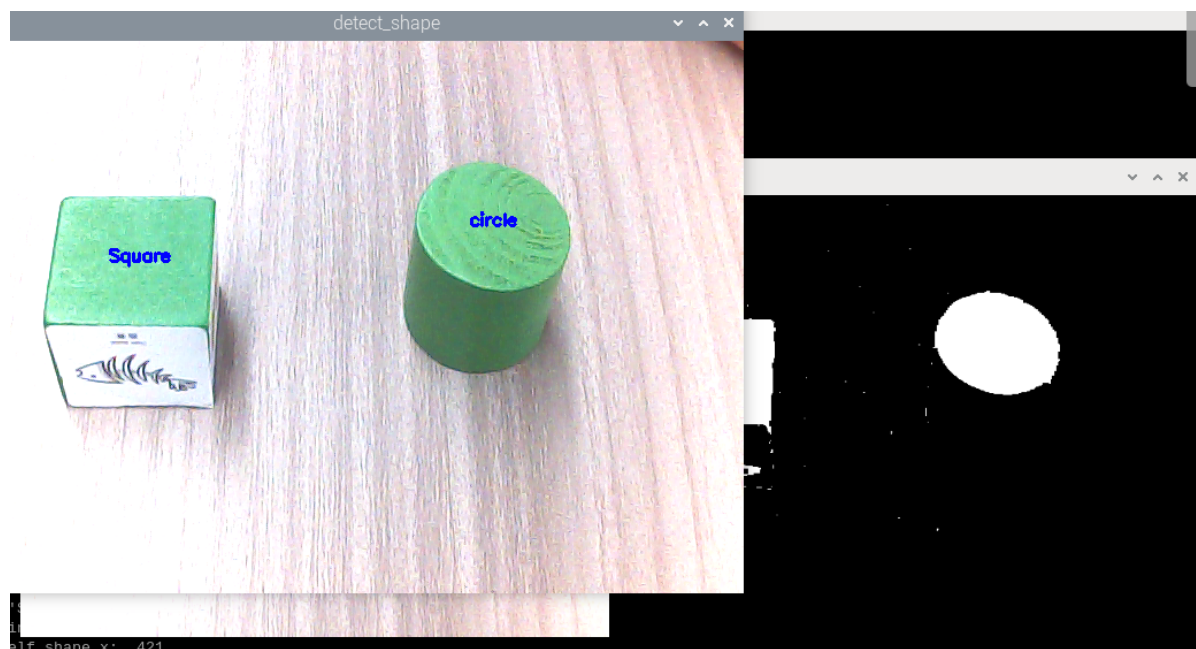
After the program runs, place blocks of the same color. After the camera captures the image, it performs color recognition based on HSV values and recognizes the shape of the block. Press the spacebar, and the robotic arm will grip the block with the target shape being square. (**Note: The default gripping width for the robotic arm should be 3cm. Gripping larger objects may cause damage to the gripper servo. This is not a quality issue.**) Place it at the set position. After placement is complete, return to the initial block recognition pose.

2. Startup and Operation

(1) Startup Commands

Enter the following commands in the terminal to start:

```
# Start camera, inverse kinematics program
ros2 launch dofbot_info camera_arm_kin.launch.py
# Start color recognition program:
ros2 run dofbot_sorting_3d shape_sorting
# Start robotic arm gripping program:
ros2 run dofbot_sorting_3d grasp
```

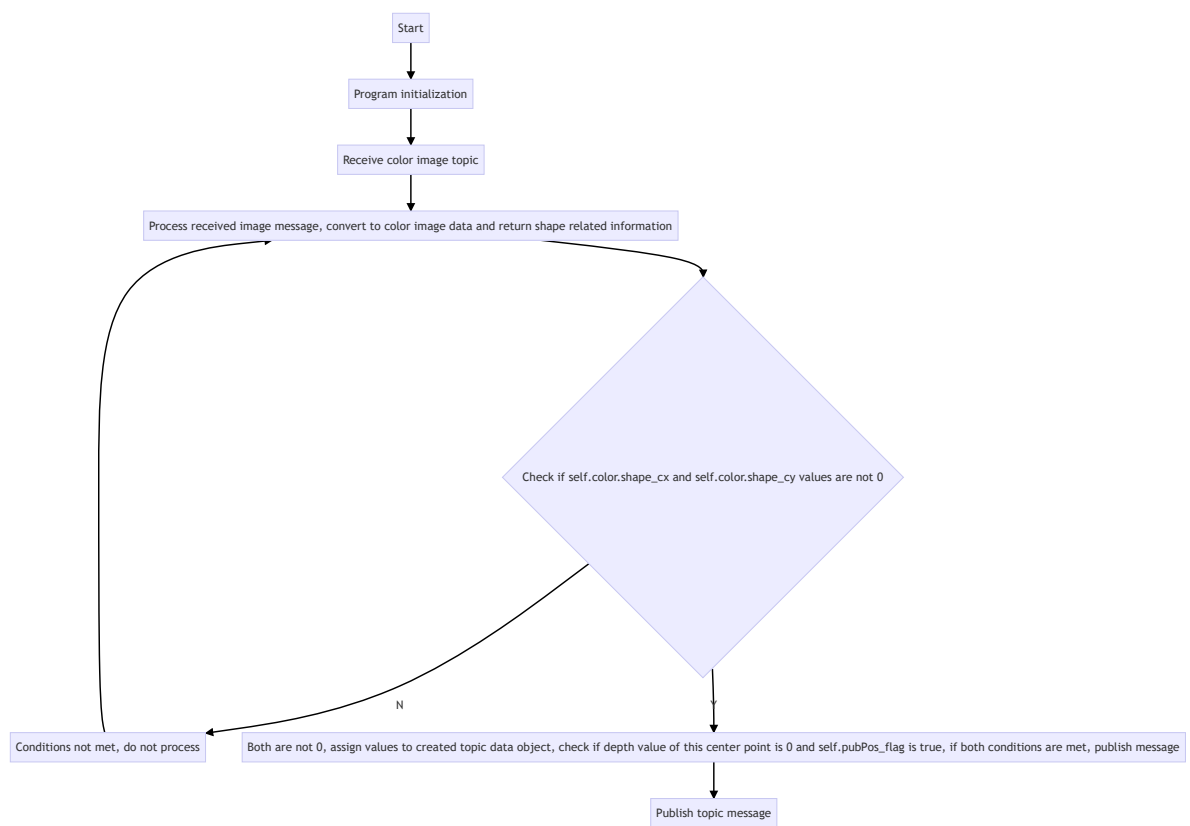


(2) Operation Process

After the terminal starts, place blocks of the same color. The camera captures the image. Press **[r]**, **[g]**, **[b]**, **[y]** to select the target block color. If recognition is not successful, you can use the mouse to select an area of the block to get the HSV values of that area. Release the mouse to enter color recognition mode. In color recognition mode, if the current HSV values still cannot filter out other colors, you can use the dynamic parameter adjuster to fine-tune the HSV values. Enter the following command in the terminal to start the dynamic parameter adjuster.

When the left image (binarized) shows only one uniquely recognized color, click on the color image frame and press the spacebar. The robotic arm will lower the gripper to grip the block with the set target shape. The default gripping target shape is square.

3. Program Flowchart



4. Core Code Analysis

shape_sorting.py

Code path:

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/shape_sorting.py
```

color_common code library path:

```
/home/yahboom/dofbot_ws/src/dofbot_sorting_3d/dofbot_sorting_3d/color_common.py
```

Import necessary libraries,

```
import cv2
import cv2 as cv
import os
import numpy as np
import time
from Arm_Lib import Arm_Device

from cv_bridge import CvBridge
from dofbot_sorting_3d.color_common import *
from dofbot_interface.msg import *
from std_msgs.msg import Bool, Int16
from rclpy.node import Node
import rclpy
import threading
```

Initialize program parameters, create publishers, subscribers, etc.,

```
def __init__(self, name):
    # Initialize node
    super().__init__(name)
    # Create arm device object
    self.Arm = Arm_Device()
    # Initial joint angles
    self.init_joints = [90, 120, 0, 0, 90, 0]
    # Create RGB image bridge object
    self.rgb_bridge = CvBridge()
```

```

# Position publish flagPosition publish flag
self.pub_pos_flag = False

# Create AprilTag position info publisherCreate AprilTag position info
publisher
self.pos_info_pub = self.create_publisher(AprilTagInfo, "PosInfo", 1)
# Create joint5 angle publisherCreate joint5 angle publisher
self.TargetJoint5_pub = self.create_publisher(Int16, "set_joint5", 10)

# Create grasp completion status subscriberCreate grasp completion status
subscriber
self.sub_grasp_status =
self.create_subscription(Bool, "grasp_done", self.get_graspStatusCallback, 100)

# Target colorTarget color
self.target_color = 0
# Red HSV threshold file pathRed HSV threshold file path
self.red_hsv_text = os.path.join(package_pwd, 'red_colorHSV.text')
# Green HSV threshold file pathGreen HSV threshold file path
self.green_hsv_text = os.path.join(package_pwd, 'green_colorHSV.text')
# Blue HSV threshold file path Blue HSV threshold file path
self.blue_hsv_text = os.path.join(package_pwd, 'blue_colorHSV.text')
# Yellow HSV threshold file path Yellow HSV threshold file path
self.yellow_hsv_text = os.path.join(package_pwd, 'yellow_colorHSV.text')
# HSV color rangeHSV color range
self.hsv_range = ()
# Region of interest initializationRegion of interest initialization
self.Roi_init = ()
# HSV range of ROI regionHSV range of ROI region
self.roi_hsv_range = ()
# Selection flagSelection flag
self.select_flags = False
# Window nameWindow name
self.windows_name = 'frame'
# Track stateTrack state
self.Track_state = 'init'
# Mouse coordinatesMouse coordinates
self.Mouse_XY = (0, 0)
# Image columns and rowsImage columns and rows
self.cols, self.rows = 0, 0
# Color detection objectColor detection object
self.color = color_detect()
# Current colorCurrent color
self.cur_color = None
# Text colorText color
self.text_color = (0,0,0)
# Circle center X coordinateCircle center X coordinate
self.cx = 0
# Circle center Y coordinateCircle center Y coordinate
self.cy = 0
# Circle radiusCircle radius
self.circle_r = 0
# Joint angle messageJoint angle message
self.joint5 = Int16()
# Corner arrayCorner array
self.corners = np.empty((4, 2), dtype=np.int32)
# Current target colorCurrent target color
self.cur_target_color = 0

```

```

# Update flagUpdate flag
self.updata_flag = False
# Set arm initial joint anglesSet arm initial joint angles
self.Arm.Arm_serial_servo_write6_array(self.init_joints,2000)
# Target distanceTarget distance
self.dist = 0.13
# Create image subscriberCreate image subscriber
self.subscription =
self.create_subscription(Image, '/image_raw', self.ImageCallback, 1)
# Target shapeTarget shape
self.target_shape = "Square"
# Shape X and Y coordinatesShape X and Y coordinates
self.shape_x = self.shape_y = 0
# Grasp completion flagGrasp completion flag
self.grasp_done = True
# Dynamic update flagDynamic update flag
self.dyn_update = False

# Declare HSV parametersDeclare HSV parameters
self.declare_parameter("Hmin", 0)
self.declare_parameter("Smin", 85)
self.declare_parameter("Vmin", 126)
self.declare_parameter("Hmax", 9)
self.declare_parameter("Smax", 253)
self.declare_parameter("Vmax", 253)

```

Let's look at the main image processing function ImageCallback,

```

def ImageCallback(self, color_frame):
    # Convert image to opencv format
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(color_frame, 'rgb8')
    rgb_image = cv2.cvtColor(rgb_image, cv2.COLOR_RGB2BGR)
    result_image = np.copy(rgb_image)
    #Space key triggers position information publishing
    key = cv2.waitKey(10)
    if key == 32:
        self.pub_pos_flag = True
        result_frame, binary = self.process(rgb_image, key)
        show_frame = threading.Thread(target=self.img_out, args=
(result_frame, binary,))
        show_frame.start()
        show_frame.join()
    # Shape recognition logic
    if self.hsv_range and self.grasp_done == True:
        # Set current target shape
        self.color.target_shape = self.target_shape
        # Perform shape recognition
        result_image, binary_processed, circle_info, corner_info =
self.color.ShapeRecognition(rgb_image, self.hsv_range)

        # Check if target shape is detected
        if self.color.shape_cx != 0 and self.color.shape_cy != 0:
            print("Find the target shape block")
            self.shape_x = self.color.shape_cx
            self.shape_y = self.color.shape_cy
            print("self.shape_x: ", self.shape_x)
            print("self.shape_y: ", self.shape_y)

```

```

        # Draw shape name on result image
        cv2.putText(result_image, self.color.shape_name,
                    (int(self.color.shape_cx) - 20, int(self.color.shape_cy)),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.5, (255, 0, 0), 2)

        if self.shape_x!=0 and self.shape_y!=0 and self.color.shape_name ==
self.target_shape:
            print(".....")
            cx = int(self.shape_x)
            cy = int(self.shape_y)
            (a, b) = (round(((320 - cx)/4000), 5), round(((480 - cy)/3000) *
0.8+0.13, 5))
            pos = AprilTagInfo()
            pos.id = self.target_color
            pos.x = float(b)
            pos.y = float(a)
            pos.z = 0.03
            if self.pub_pos_flag == True:
                print("-----")
                self.pub_pos_flag = False
                self.pos_info_pub.publish(pos)
                print(f"Published position info: ID={pos.id}, x={pos.x}, y=
{pos.y}, z={pos.z}")
                print("*****")
                self.shape_x = 0
                self.shape_y = 0
                self.grasp_done = False
            cv.imshow("detect_shape", result_image)

```

Image processing function self.process,

```

def process(self, rgb_img, key):
    # Resize image to 640x480
    rgb_img = cv.resize(rgb_img, (640, 480))
    # Initialize binary image list
    binary = []
    # Press C key: clear target color and reset
    if key == ord('c') or key == ord('C'):
        self.target_color = 0
        self.Reset()
        self.updata_flag = True
    # Press R key: set target color to red
    elif key == ord('r') or key == ord('R'):
        self.target_color = 3
        self.cur_target_color = self.target_color
    # Press G key: set target color to green
    elif key == ord('g') or key == ord('G'):
        self.target_color = 2
        self.cur_target_color = self.target_color
    # Press B key: set target color to blue
    elif key == ord('b') or key == ord('B'):
        self.target_color = 1
        self.cur_target_color = self.target_color

```

```

# Press Y key: set target color to yellow Press Y key: set target color to
yellow
elif key == ord('y') or key == ord('Y'):
    self.target_color = 4
    self.cur_target_color = self.target_color
# Press I key or target color exists: enter identify mode Press I key or
target color exists: enter identify mode
elif key == ord('i') or key == ord('I') or self.target_color!=0:
self.Track_state = "identify"
# Print track state for debugging Print track state for debugging
#print("self.Track_state: ",self.Track_state)
# Init state: create window and set mouse callback Init state: create window
and set mouse callback
if self.Track_state == 'init':
    cv.namedWindow(self.windows_name, cv.WINDOW_AUTOSIZE)
    cv.setMouseCallback(self.windows_name, self.onMouse, 0)
    # If region is selected, draw selection box If region is selected, draw
selection box
    if self.select_flags == True:
        cv.line(rgb_img, self.cols, self.rows, (255, 0, 0), 2)
        cv.rectangle(rgb_img, self.cols, self.rows, (0, 255, 0), 2)
        # If selection region is valid, calculate HSV range If selection
region is valid, calculate HSV range
        if self.Roi_init[0] != self.Roi_init[2] and self.Roi_init[1] !=
self.Roi_init[3]:
            rgb_img, self.hsv_range = self.color.Roi_hsv(rgb_img,
self.Roi_init)
            # Save HSV range of ROI region Save HSV range of ROI region
            self.roi_hsv_range = self.hsv_range
            # Set dynamic update flag Set dynamic update flag
            self.dyn_update = True
        else: self.Track_state = 'init'

# Identify state: load HSV threshold based on target color Identify state:
load HSV threshold based on target color
elif self.Track_state == "identify":
    # Red: load red HSV threshold Red: load red HSV threshold
    if self.target_color == 3:
        self.hsv_range = read_HSV(self.red_hsv_text)
        self.cur_color = "red"
        self.text_color = (0, 0, 255)

# Green: load green HSV threshold Green: load green HSV threshold
elif self.target_color == 2:
    self.hsv_range = read_HSV(self.green_hsv_text)
    self.cur_color = "green"
    self.text_color = (0, 255, 0)

# Blue: load blue HSV threshold Blue: load blue HSV threshold
elif self.target_color == 1:
    self.hsv_range = read_HSV(self.blue_hsv_text)
    self.cur_color = "blue"
    self.text_color = (255, 0, 0)

# Yellow: load yellow HSV threshold Yellow: load yellow HSV threshold
elif self.target_color == 4:

```

```

        self.hsv_range = read_HSV(self.yellow_hsv_text)
        self.cur_color = "yellow"
        self.text_color = (255, 255, 0)

        # Invalid color: return to init state Invalid color: return to init
state
    else:
        self.Track_state = 'init'

        # If not in init state, perform object tracking If not in init state, perform
object tracking
        if self.Track_state != 'init':
            # If HSV range is valid, perform object tracking If HSV range is valid,
perform object tracking
            if len(self.hsv_range) != 0:
                rgb_img, binary, self.circle,_,self.corners=
self.color.object_follow(rgb_img, self.hsv_range)
                # If dynamic update is needed, save HSV threshold to file and update
ROS parameters
                # If dynamic update is needed, save HSV threshold to file and update
ROS parameters
                if self.dyn_update is True:
                    # Write HSV threshold to corresponding file based on current
target color Write HSV threshold to corresponding file based on current target
color

                    if self.target_color == 3:
                        write_HSV(self.red_hsv_text, self.hsv_range)
                    elif self.target_color == 2:
                        write_HSV(self.green_hsv_text, self.hsv_range)
                    elif self.target_color == 1:
                        write_HSV(self.blue_hsv_text, self.hsv_range)
                    elif self.target_color == 4:
                        write_HSV(self.yellow_hsv_text, self.hsv_range)

                    # Update ROS parameters Update ROS parameters
                    self.Hmin = rclpy.parameter.Parameter('Hmin',
rclpy.Parameter.Type.INTEGER, int(self.roi_hsv_range[0][0]))
                    self.Smin = rclpy.parameter.Parameter('Smin',
rclpy.Parameter.Type.INTEGER, int(self.roi_hsv_range[0][1]))
                    self.Vmin = rclpy.parameter.Parameter('Vmin',
rclpy.Parameter.Type.INTEGER, int(self.roi_hsv_range[0][2]))
                    self.Hmax = rclpy.parameter.Parameter('Hmax',
rclpy.Parameter.Type.INTEGER, int(self.roi_hsv_range[1][0]))
                    self.Smax = rclpy.parameter.Parameter('Smax',
rclpy.Parameter.Type.INTEGER, int(self.roi_hsv_range[1][1]))
                    self.Vmax = rclpy.parameter.Parameter('Vmax',
rclpy.Parameter.Type.INTEGER, int(self.roi_hsv_range[1][2]))
                    all_new_parameters = [self.Hmin, self.Smin, self.Vmin,
self.Hmax, self.Smax, self.Vmax]
                    self.set_parameters(all_new_parameters)
                    # Reset dynamic update flag Reset dynamic update flag
                    self.dyn_update = False

                # Save circle center coordinates and radius Save circle center
coordinates and radius
                self.cx = self.circle[0]
                self.cy = self.circle[1]
                self.circle_r = self.circle[2]

```

```

        # Save HSV threshold to file based on current target color Save HSV
        threshold to file based on current target color
        if self.cur_target_color == 3 and self.updata_flag == True:
            write_HSV(self.red_hsv_text, self.hsv_range) # 红色 Red
        elif self.cur_target_color == 2 and self.updata_flag == True:
            write_HSV(self.green_hsv_text, self.hsv_range) # 绿色 Green
        elif self.cur_target_color == 1 and self.updata_flag == True:
            write_HSV(self.blue_hsv_text, self.hsv_range) # 蓝色 Blue
        elif self.cur_target_color == 4 and self.updata_flag == True:
            write_HSV(self.yellow_hsv_text, self.hsv_range) # 黄色 Yellow
        # Reset update flag Reset update flag
        self.updata_flag = False

    # Draw current color name on image Draw current color name on image
    rgb_img = cv2.putText(rgb_img, self.cur_color, (10, 30),
cv2.FONT_HERSHEY_SIMPLEX, 1, self.text_color, 2)
    # Return processed image and binary image Return processed image and binary
    image
    return rgb_img, binary

```

grasp.py

The previous chapter has explained this, so I won't repeat it here